



Figure 4: LTtngTop

The interface is quite intuitive, and the event and the control flow views are synchronised. Next, click on the 'Resources' tab and you will see all the system resources and their consumption during the trace. See Figure 2(b). Of course, it's a lot better this way rather than looking at *cpu_id* each time in the *babeltrace* output. You can also have a look at the 'Statistics' and 'Histogram' tab, which gives you a more comprehensive view of the system's functioning so that you can observe trends statistically too.

Now that you are familiar with the interface, do a small activity by which you try to see what happens in the system when the userspace tracepoint was hit and recorded the *count* data from *handle_newconnect()* in *thttpd*. You can create a *trace experiment* for this. Go to *Project explorer* → *Right click Experiments* → *New (name it 'Combined')* → *Select kernel and ust traces*. Now double click on 'Combined' and there you go. You have combined UST and kernel traces now. Interesting! Now you can know exactly how the system behaved while the userspace tracepoint was hit. Under the *<srch>* in *Type*, write *'thttpd:count'* and your first UST tracepoint that was hit shall be displayed. Above and below that, the system activity in kernel trace will be shown too. See Figure 3.

There are other CTF trace viewers out there too. Another graphical one is the Linux Trace Toolkit Viewer (LTTV), which you can install by using the following command:

```
$ sudo yum install lttv
```

Being a precursor to TMF, it was one of the original graphical trace viewers and inspired a lot of techniques and methods that were used in TMF. LTTV is quite fast but may not be as versatile and comprehensive as TMF.

Looking ahead

LTtng 2.4 'Époque Opaque' is around the corner (RC5 is available at the time of writing) and will have some new and awesome features:

1. *Live trace viewing*: Yes, you heard that right. You can

now view and analyse your traces while they are being created. You can view the live streaming data on any remote machine too. There is another tool, LTtngTop (it's an *ncurses*-based tool like viewer), for live as well as already recorded traces which you can use with this feature. See Figure 4 and check it out at <http://git.lttng.org/?p=lttngtop.git>. Live trace reading will soon come to Eclipse TMF also (maybe not in the next release). This is a feature that sysadmins would really love!

2. *Snapshots*: Soon you will also be able to take a 'snapshot' of the trace buffers at a given time and observe the traced data. It's a pretty cool feature for scenarios in which you may want to trigger a snapshot record at some coredump, for example. Snapshot trace data can also be used with flight-recorder mode (the trace data snapshot gets overwritten in the buffer as in a flight recorder). Long term system monitoring is also a good use-case for snapshots.

The fancy control flow view is fine, but you may wonder what the actual path a process takes through the system during execution is. Can you show that? Indeed, it may soon be a reality. As for the question we discussed earlier about APT taking up your valuable time while installing packages, *Critical Flow View* answers that. Refer to this awesome post at <http://multivax.blogspot.ca/2013/12/system-level-profiling-of-apt.html>.

I would also recommend that you check out Workload Kit (<https://github.com/giraldeau/workload-kit>). It is a way to quickly generate repeatable workloads for tracing on your machines. It contains *ltng-simple*, which is a command line tool to quickly start with LTtng on your system and do some experimentation.

This concludes the series on tracing using the LTtng tools. In case you have any questions, do contact me.

Happy tracing! 

References

- [1] <http://lttng.org/files/doc/man-pages/man3/lttng-ust.3.html>
- [2] <http://secretaire.dorsal.polymtl.ca/~alexmont/tracing-rp/>
- [3] <http://git.lttng.org/?p=lttngtop.git>
- [4] <http://multivax.blogspot.ca/2013/12/system-level-profiling-of-apt.html>
- [5] <https://github.com/giraldeau/workload-kit>

Acknowledgements

Thanks to Geneviève Bastien and François Rajotte for reviewing this article and to all the LTtng developers for their contributions. The TMF dev team at Ericsson and Polytechnique Montréal deserve special kudos!

By: Suchakrapani Sharma

The author is a PhD student at École Polytechnique de Montréal. He is currently doing research on dynamic tracing tools. He has varied interests, from performance analysis tools to Embedded Linux and UX/graphics design. He can be contacted at suchakra@fedoraproject.org